

AI ASSISTED CODING

END LAB EXAM

Name: K.ShivaShankar

Roll Number: 2403A510E5

Batch: 24BTCAICSB05

SET 6

Question 1:

Auto-generate API docs

Task 1: Use AI to generate Markdown documentation.

Prompt:

Use AI or automated static analysis to generate Markdown-formatted API documentation from Python source code. Extract function names, parameters, and existing docstrings, and convert them into clean developer-friendly

Code-generated:

```
import ast

def generate_markdown_from_code(source_code: str) -> str:
    tree = ast.parse(source_code)
    md = "# API Documentation\n\n"

    for node in tree.body:
        if isinstance(node, ast.FunctionDef):
            md += f"## `{node.name}`\n\n"

            # Docstring
            doc = ast.get_docstring(node)
            if doc:
                md += f"{doc}\n\n"

            # Parameters
            md += "**Parameters:**\n"
            if node.args.args:
                for arg in node.args.args:
```

```

        md += f"- `{arg.arg}`\n"
    else:
        md += "- None\n"
    md += "\n"

    # Returns
    md += "**Returns:**\n"
    md += "- Unknown (static analysis)\n\n"

    return md

# Example usage
source = '''
def multiply(x, y):
    """Multiply two numbers."""
    return x * y
'''

print("=== AUTO-GENERATED MARKDOWN DOCS ===")
print(generate_markdown_from_code(source))

```

Output:

```
=== AUTO-GENERATED MARKDOWN DOCS ===  
# API Documentation  
  
## `multiply`  
Multiply two numbers.  
  
**Parameters:**  
- `x`  
- `y`  
  
**Returns:**  
- Unknown (static analysis)
```

Observation:

This script analyzes the Python code using the ast module to statically extract function information. It structures the extracted metadata into well-organized Markdown suitable for documentation. The tool does not need external libraries and focuses on lightweight static code parsing.

Task 2: Add auto-generated code comments

Prompt:

Automatically add descriptive docstrings to Python functions that lack comments. Detect functions with missing documentation and insert meaningful auto-generated docstrings at the beginning of each function body.

Code generated:

```
import ast

def add_docstrings(source_code: str) -> str:
    tree = ast.parse(source_code)

    for node in tree.body:
        if isinstance(node, ast.FunctionDef):
            # Add only if missing
            if not ast.get_docstring(node):
                auto_comment = f"Auto-generated
docstring for `{node.name}`."
                node.body.insert(0,
ast.Expr(ast.Constant(auto_comment)))

    return ast.unparse(tree)

# Example usage
source = '''
def add(a, b):
    return a + b

def hello(name):
    return f"Hello, {name}"
'''

print("=== CODE WITH AUTO-GENERATED COMMENTS ===")
print(add_docstrings(source))
```

Output:

```
=== CODE WITH AUTO-GENERATED COMMENTS ===
def add(a, b):
    """Auto-generated docstring for `add`."""
    return a + b

def hello(name):
    """Auto-generated docstring for `hello`."""
    return f'Hello, {name}'
PS C:\Users\HP> 
```

Observation:

This script scans Python code and identifies functions that lack docstrings by using the AST parser. When a missing docstring is found, it inserts an auto-generated comment inside the function body.

Question 2: Create user manual

Task 1: Use AI to convert technical descriptions to simple language.

Prompt:

Use AI to rewrite technical software or hardware descriptions in simple, everyday language for non-technical users. Ensure the meaning remains accurate while removing jargon, simplifying terms, and improving clarity.

Code generated:

```
def simplify_text(technical_text: str) -> str:
    # NOTE: In real usage, this would call an AI
    model.
    # Here we simulate simplification using keyword
    replacements.
    replacements = {
        "initialize": "start",
        "configure": "set up",
        "execute": "run",
        "terminate": "stop",
        "authentication": "login",
```

```

        "synchronously": "at the same time",
        "asynchronously": "in the background"
    }

    simplified = technical_text
    for tech, simple in replacements.items():
        simplified = simplified.replace(tech, simple)

    return simplified

# Example usage
text = "To initialize the system, authenticate the
user and configure all modules asynchronously."
print("=== SIMPLIFIED USER-FRIENDLY INSTRUCTIONS
===")
print(simplify_text(text))

```

Output:

```

=== SIMPLIFIED USER-FRIENDLY INSTRUCTIONS ===
To start the system, authenticate the user and set up all modules aat the same time.

```

Observation:

This script imitates AI-driven text simplification using a dictionary of technical-to-simple replacements. Although basic, it demonstrates how

complex terminology can be transformed into beginner-friendly language.

Task 2: Create step-by-step screenshots description.

Prompt:

Use AI to generate clear, step-by-step descriptions of screenshots for a user manual. Each step should describe what the user sees, what button to click, and what happens next. The instructions should be visual, easy to follow, and written for beginners.

Code generated:

```
def describe_screenshots(steps: list) -> str:
    description = "## Step-by-Step Screenshot\n\nGuide\n\n"
    for i, step in enumerate(steps, 1):
        description += f"### Step {i}\n"
        description += f"- **Screenshot shows:**\n\n{step['view']}\n"
```

```

        description += f"- **User should:**\n"
    {step['action']}\n"
        description += f"- **Result:**\n\n"
    {step['result']}\n\n"
    return description
# Example usage
steps_data = [
    {
        "view": "The homepage with a large blue
'Login' button in the center.",
        "action": "Click the 'Login' button.",
        "result": "You are taken to the login
screen."
    },
    {
        "view": "The login page with fields for email
and password.",
        "action": "Enter your email and password,
then press 'Submit'.",
        "result": "You are logged into your
dashboard."
    }
]
print("=== STEP-BY-STEP SCREENSHOT DESCRIPTION ===")
print(describe_screenshots(steps_data))

```

Output:

```
=== STEP-BY-STEP SCREENSHOT DESCRIPTION ===  
## Step-by-Step Screenshot Guide  
  
### Step 1  
- **Screenshot shows:** The homepage with a large blue 'Login' button in the center.  
- **User should:** Click the 'Login' button.  
- **Result:** You are taken to the login screen.  
  
### Step 2  
- **Screenshot shows:** The login page with fields for email and password.  
- **User should:** Enter your email and password, then press 'Submit'.  
- **Result:** You are logged into your dashboard.
```

Observation:

This script structures screenshot descriptions into a formatted, sequential user manual section. Each step contains a visual description, user action, and expected outcome for clarity. Although simulated, this mirrors how an AI system would generate instructional walkthroughs.